

Faster Logical Operations from Discrete CKKS

Jaehyung Kim

Stanford University, Stanford, USA
jaehk@cs.stanford.edu

Abstract. We study efficient non-arithmetic operations in (G)BFV with arbitrary plaintext modulus. We design scheme conversions between (G)BFV and Discrete CKKS, allowing (G)BFV ciphertexts to use radix-based integer computation in the Discrete CKKS setting. This gives asymptotically faster logical operations: for plaintext modulus p , homomorphic comparison runs in $O(\log p \log \log p)$ for BFV and $O(\log \log p)$ for GBFV.

Keywords: GBFV · BFV · Discrete CKKS · Logical Operations

1 Introduction

The state-of-the-art fully homomorphic encryption schemes can be categorized into three families: The BGV [BGV12]/BFV [Bra12,FV12] family for SIMD integer arithmetic, the DM [DM15]/CGGI [CGGI16] family for low-latency computations, and the CKKS [CKKS17] family for real number arithmetic. Among the three, the BGV/BFV family has been considered as the primary option for integer-heavy computations such as private information retrieval (PIR) [MCR21] and private set intersection (PSI) [CLR17].

Although the BGV/BFV family natively supports addition and multiplication, non-arithmetic operations such as comparison, digit decomposition, and bit-wise operations often require expensive polynomial interpolation. For a prime modulus p , a univariate interpolation of degree p can evaluate an arbitrary function, which might be prohibitively expensive for large p . The standard way to overcome to $O(p)$ computational barrier is to use a digit-decomposed representation of integers. For instance, the state-of-the-art homomorphic comparison for BGV/BFV [IZ21,TLW⁺21] uses a representation $x = \sum_{i=0}^{d-1} p^i \cdot x_i$ where $x_i \in [0, p)$ for each $0 \leq i < d$. The main problem of this kind of approach is that the digit-decomposed format is not well compatible with arithmetic operations. A natural approach [ZCL24] is to build a conversion between the regular $\mathbb{Z}_{p^d}$ representation and the digit representation $\mathbb{Z}_p^d$. As a result, one can achieve a logarithmic asymptotic complexity ($O(d)$) for a plaintext modulus p^d .

This line of approach, however, requires a plaintext modulus to be a power of a small prime. Many applications, such as homomorphic signing [BGG⁺18,BK26], FHE-SNARK [ZWL⁺25], and SPDZ matrix tuple generation [CHM⁺25], require the plaintext modulus to be a large prime. In addition, (NTT-friendly) prime modulus is essential for full SIMD support, as the ring $\mathcal{R}_p$ needs to be fully split. Hence we ask a natural question:

Can we have a framework that supports non-arithmetic operations for BGV/BFV with arbitrary plaintext modulus?

One recent direction toward this goal is the functional bootstrapping framework of Lee, Min, and Song [LMS24], which evaluates arbitrary functions during bootstrapping by reducing them to LUT evaluation over prime-power plaintext spaces. Their approach provides a general functional-bootstrapping framework for BFV based on LUT evaluation, with optimized treatment of Heaviside/sign-style functions such as sign and delta. In this work, we pursue a different direction based on Discrete CKKS [DMPS24, BCKS24, BKSS24, AKP25]. Rather than realizing each target function through LUT evaluation during bootstrapping, we design an efficient scheme conversion from BFV to Discrete CKKS and use the resulting radix-based representation as a reusable computational backend for non-arithmetic operations such as comparison and bit-wise computation. The conversion is based on the iterative MSB bootstrapping from [KN25] and the Discrete CKKS-based integer computer [Kim25b], and its asymptotic complexity is roughly logarithmic in the plaintext modulus p . Furthermore, we extend this discussion to the generalization of BFV called GBFV [GV25, CHM⁺25], where the asymptotic complexity of the conversion is reduced to $O(\log \log p)$ with the help of the homomorphic integer computer from [CPL25].

1.1 Technical Overview

Let N be a power-of-two integer and q be a ciphertext modulus. Let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$. We start from a coefficient-encoded BFV ciphertext encrypting $m \in \mathcal{R}_p$ with plaintext modulus p . This can be written as

$$\text{ct} = (b, a) = \left(-as + \left\lfloor \frac{q}{p} \cdot m \right\rfloor + e, a \right)$$

where $s \in \mathcal{R}$ is a secret key. In other words, ct decrypts to

$$\text{ct} \cdot (1, s) = \left\lfloor \frac{q}{p} \cdot m \right\rfloor + e.$$

as a RLWE ciphertext. The conversion from BFV to Discrete CKKS can be described as follows:

1. **Modulus Raising:** When we modraise ct , we get

$$\text{ModRaise}(\text{ct}) \cdot (1, s) = \left\lfloor \frac{q}{p} \cdot m \right\rfloor + e + qI$$

for some small integral polynomial $I \in \mathcal{R}$. Our key observation is to regard this as a noisy Discrete CKKS ciphertext encrypting $m + qI$ with scaling factor q/p . In this case, we may write

$$\text{ModRaise}(\text{ct}) \cdot (1, s) = \lfloor \Delta \cdot (m + qI) \rfloor + e.$$

for $\Delta = q/p$.

2. **Iterative MSB Bootstrapping:** The iterative MSB bootstrapping in [KN25] decomposes a Discrete CKKS ciphertext into digits. Let

$$m + pI = \sum_{i=0}^k b^i \cdot m_i$$

be the base b digit decomposition of $m + qI$. The result of the MSB bootstrapping are k discrete ciphertexts $\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{k-1}$ where ct_i is an encryption of m_i for each $0 \leq i < k$.¹ This type of encoding is exactly the encoding of the integer computer in [Kim25b].

3. **Homomorphic Modular Reduction:** Since [Kim25b] supports addition, subtraction, multiplication, comparison, and shift operations, we can simulate homomorphic modular reduction by p inside this homomorphic computer. We may use either Montgomery reduction or base reduction, as [CPL25] did in a different homomorphic computer. The result is a vector of ciphertexts $\text{ct}'_0, \text{ct}'_1, \dots, \text{ct}'_{k-1}$ encrypting m with respect to the encoding of [Kim25b]. In terms of asymptotic cost, it involves at least one homomorphic multiplication which is $O(k \log k) = O(\log p \log \log p)$. This step finishes a conversion from BFV to [Kim25b].

The conversion from BFV to [Kim25b] by definition gives an efficient homomorphic digit decomposition. Recall that directly achieving this inside BFV requires $O(p)$ whereas our construction requires $O(\log p \log \log p)$ which is sublinear. A digit-decomposed representation directly provides efficient non-arithmetic operations. For instance, a comparison can be instantiated as (1) converting two BFV ciphertexts into [Kim25b], (2) running comparison inside [Kim25b], and (3) converting back to BFV. Note that the conversion from Discrete CKKS to BFV is very cheap: One can simply clean (if needed), perform Slots-to-Coefficients, and take modular reduction. Overall, homomorphic comparison complexity reduces from $O(p)$ to $O(\log p \log \log p)$. Other non-arithmetic operations can be accelerated in a similar manner, and we discuss this in detail in the later sections.

We extend this observation to GBFV [GV25, CHM⁺25]. Let k be a power-of-two integer, $b > 1$ be an integer, and $t(X) = X^k - b$. A GBFV ciphertext encrypting a message $m \in \mathcal{R}_t = \mathcal{R}/t\mathcal{R}$ can be written as

$$\text{ct} = (b, a) = (-as + \lfloor (q/t) \cdot m \rfloor + e, a)$$

for a secret key s . In other words, ct decrypts to

$$\text{ct} \cdot (1, s) = \lfloor (q/t) \cdot m \rfloor + e$$

as a RLWE ciphertext. When we modraise ct , we get

$$\text{ModRaise}(\text{ct}) \cdot (1, s) = \lfloor (q/t) \cdot m \rfloor + e + qI$$

¹ Since the size of I is $O(1)$, we have $k = \log_b p + O(1)$.

for some small integral polynomial $I \in \mathcal{R}$. We multiply by t and get ct' such that

$$\text{ct}' \cdot (1, s) = q \cdot (m + tI) + e'.$$

where $e' \approx te$ is small. We observe that $m + tI$ is an approximate digit decomposition of m modulo $p = b^{N/k} + 1$. The ciphertexts in [CPL25] decompose a large integer into approximate digits, which is essentially the same encoding up to reordering. We use RLWE-to-LWE extraction and ring packing from [BCK⁺23] to convert the ciphertext ct' into a [CPL25] ciphertext encrypting the same message m modulo p . As soon as the encoding structure is matched, we can use the homomorphic modular reduction in [CPL25] to modular reduce by p and obtain an exact digit-decomposed form of m . Once we have the full conversion from GBFV to [CPL25], we can similarly accelerate logical operations for GBFV. The asymptotic complexity of the conversion is the same as the homomorphic modular reduction complexity, which is $O(\log \log p)$.

1.2 Contributions

The main contribution of our paper is a framework for asymptotically faster logical operations in (G)BFV, obtained by introducing efficient conversions between (G)BFV and Discrete CKKS. Along the way, we enrich the Discrete CKKS toolkit by introducing an efficient arbitrary-modulus modular reduction. In addition, we show how sparse packing can be exploited to further improve efficiency.

Efficient (G)BFV to Discrete CKKS conversion. We efficiently bridge (G)BFV and Discrete CKKS, allowing (G)BFV to benefit from efficient radix-based computation for arbitrary plaintext modulus. For a plaintext modulus p , the asymptotic conversion complexity is $O(\log p \log \log p)$ or $O(\log \log p)$, which is fully practical, roughly on the scale of a few (G)BFV bootstrappings. Poor support for logical and other non-arithmetic operations has long been one of the main weaknesses of BGV/BFV, and our framework provides a simple and generic solution to this problem. In particular, our conversion framework enables BFV to support homomorphic comparison with complexity $O(\log p \log \log p)$ for general modulus p , improving the asymptotic complexity over prior BFV comparison methods for general modulus p . In the case of GBFV, the complexity further decreases to $O(\log \log p)$.

CKKS modular reduction for arbitrary modulus. The key component of our BFV-to-Discrete-CKKS conversion is a new MSB bootstrapping for arbitrary modulus. Recall that the MSB bootstrapping from [KN25] iterates $\mathbb{Z}_b$ functional bootstrapping k times to bootstrap $\mathbb{Z}_{b^k}$ in the most significant bits. If one wants to bootstrap $\mathbb{Z}_p$ for a general modulus p , it should choose iteration number 1 and have complexity $O(p)$. Instead of directly running functional bootstrapping, we modulus-raise, decompose, and modular reduce inside the homomorphic integer framework of [Kim25b], dropping the asymptotic complexity

to $O(\log p \log \log p)$. Since MSB bootstrapping is equivalent to homomorphic modular reduction, we get an arbitrary modulus homomorphic modular reduction for CKKS, significantly improving the performance of the homomorphic modular reduction in [KN25].

Faster Logical Operations for Sparse Packing. If BFV ciphertexts are sparsely packed, we reduce the conversion complexity further down to (almost) $O(\log \log p)$, by providing an efficient conversion to [CPL25]. Similarly in CKKS, sparsely packed ciphertexts can achieve asymptotically faster homomorphic comparison, decreasing from $O(k)$ to $O(\log k)$ for comparison precision bit-length k .

1.3 Related Works

Since the first instantiation of FHE [Gen09], there have been numerous works in improving the performance of FHE. The BFV scheme, introduced by Brakerski, Fan, and Vercauteren [Bra12,FV12], provides a conceptually simple homomorphic integer computations by putting message in the most significant bits, separating from the error accumulating from the least significant bits. The scheme by Chen et al. [CLPX18] chooses the plaintext modulus as a polynomial $t(X) = X - b$ instead of an integer (which is the case for BFV) and achieved high precision computation with significantly less noise growth. However, their construction was used in limited scenarios as it was not bootstrappable and lacks SIMD capability which is one of the biggest strength of BFV. Recently, two independent works [GV25,CHM⁺25] extended the plaintext space to a general ideal (so that $t(X)$ becomes any polynomial), and achieved a promising middle ground between BFV and CLPX. In particular, [GV25] instantiated the first bootstrapping for this type of schemes.

The CKKS scheme by Cheon et al. [CKKS17], is often considered as the most promising scheme for privacy preserving machine learning, as it provides efficient SIMD real number arithmetic. One of the recent major improvements in the CKKS domain is a variant of CKKS that handles discrete computations, suggested by Drucker et al. [DMPS24]. The heart of Discrete CKKS is a CKKS-style functional/programmable bootstrapping [BCKS24,BKSS24,AKP25] which evaluates an arbitrary function during bootstrapping. Based on this framework, several works [Kim25b,BK25,Kim25c,CPL25] have built integer computation units, providing efficient high-precision integer computations including logical operations.

Since different schemes are strong for different capabilities, the notion of scheme conversion have been widely adopted in the literature. For instance, Chimera [BGGJ20] and Pegasus [jLHH⁺21] propose a scheme switching framework for the entire FHE domain including conversions between LWE and RLWE ciphertexts. However, these conversions do not discuss an efficient conversion from BFV to CKKS and vice versa. More relevant conversion frameworks are (G)BFV bootstrappings from CKKS [KSS24,Kim25a] which regard (G)BFV error as a CKKS ciphertext and leverage CKKS bootstrapping to bootstrap

(G)BFV. In particular, [Kim25a] introduces a conversion from GBFV to CKKS and vice versa. However, the conversion is not exact in the sense that it does not convert to the correct digit decomposition but a redundant decomposition: Each component is not in $[0, b)$ but in $[0, cb)$ for some constant c .

Since BFV primarily supports only addition and multiplication, achieving non-arithmetic operations like comparison has been challenging. Since a prime field of size p can evaluate any univariate function in degree $p - 1$ polynomial interpolation, the straightforward approach is to rely on this interpolation which is very costly. The main attempts were focused on starting from a decomposed format [IZ21, TLW⁺21]: relying on already decomposed integers with a digit base p which is a small prime. In order to connect this framework with arithmetic operations like addition and multiplication, [ZCL24] introduced an asymptotically efficient conversion from $\mathbb{Z}_{p^d}$ to $\mathbb{Z}_p^d$ to bridge two different types of encodings.

In [LMS24], the authors introduce a functional bootstrapping framework for BFV that supports plaintext-modulus switching. More precisely, they start from a BFV ciphertext with plaintext modulus t , reinterpret it via modulus switching as a noisy encryption over an intermediate ring $\mathbb{Z}_{p^r}$ for a suitable r , and then evaluate a LUT by functional bootstrapping. Their Heaviside/sign-style LUT evaluation uses $O(r^{3/2})$ key-switchings; equivalently, for fixed base prime p , the asymptotic complexity with respect to the input plaintext modulus t is roughly $O((\log t)^{3/2})$. Their framework outputs a ciphertext over $\mathbb{Z}_{p^s}$, and does not provide a general exact conversion back to the original plaintext modulus t .

Concurrent Work. A concurrent and independent work [PPS26] homomorphically evaluates Dilithium [DKL⁺18] to achieve its threshold version. Along the way, they propose a CKKS-BFV continuum, which is a framework that uses conversions between Discrete CKKS and BFV for better efficiency. In particular, [PPS26, Section 4.4] can be understood as our conversion from BFV to CKKS in Section 3 with bit-decomposed representation (i.e, $b = 2$). Our paper addresses the technique in more detail and further generalizes the framework to GBFV and sparse BFV/CKKS.

2 Preliminaries

Let $N \geq 2$ be a power-of-two integer and $q > 0$ be an integer. Let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$. For an integral polynomial $\mu(X)$, we say that $\alpha \leq \mu < \beta$ if all of its coefficients are in $[\alpha, \beta)$.

2.1 BFV and GBFV Basics

Let $t \in \mathcal{R}$ be a polynomial and $\mathcal{R}_t = \mathcal{R}/t\mathcal{R}$. For simplicity, we only consider the case $t(X) = p$ for some $p \in \mathbb{Z}_{>0}$ and $t(X) = X^k - b$ for some power-of-two integer k and $b \in \mathbb{Z}$. Each type represents BFV and GBFV, respectively. In this case,

$$\frac{1}{t} = \frac{1}{p} \text{ or } -\frac{b^{N/k-1} + b^{N/k-2}X^k + \dots X^{N-k}}{b^{N/k} + 1} \in \mathbb{Q}[X]$$

so for any polynomial $u(X) \in \mathcal{R}$, the rounded polynomial

$$\lfloor u(X)/t(X) \rfloor \in \mathcal{R}$$

is well defined. For simplicity, we consider a coefficient-encoded variants of BFV and GBFV, whose plaintext spaces are $\mathcal{R}_t$ rather than the SIMD space $\mathbb{F}_{p^d}^\ell$ for some d and ℓ .

Encryption and Decryption. Given a plaintext $m \in \mathcal{R}_t$, the encryption of m under the secret key $\text{sk} = (1, s) \in \mathcal{R}^2$ is defined as

$$\text{GBFV.Enc}(\text{sk}, m) = (-as + \lfloor \Delta \cdot m \rfloor + e, a) \in \mathcal{R}_q^2$$

where a is uniformly sampled from $\mathcal{R}_q$, $\Delta = q/t$, and e is sampled from a noise distribution. A (G)BFV ciphertext is of the form

$$(-as + \lfloor \Delta \cdot m \rfloor + e, a) \in \mathcal{R}_q^2,$$

where m is the underlying plaintext and e is the underlying noise. Given a ciphertext $\text{ct} \in \mathcal{R}_q^2$, the decryption with respect to the secret sk is defined as

$$\text{GBFV.Dec}(\text{sk}, \text{ct}) = \left\lfloor \frac{\text{ct} \cdot \text{sk}}{\Delta} \right\rfloor \in \mathcal{R}_t.$$

The decryption succeeds only when the underlying noise e is small enough.

Homomorphic Operations. As a black box, (G)BFV supports homomorphic addition and multiplication.² That is, given two ciphertexts $\text{ct}_1, \text{ct}_2 \in \mathcal{R}_q^2$ encrypting $m_1, m_2 \in \mathcal{R}_t$, respectively, addition and multiplication denoted as

$$\text{GBFV.Add}(\text{ct}_1, \text{ct}_2) \text{ and } \text{GBFV.Mult}(\text{ct}_1, \text{ct}_2)$$

satisfy

$$\text{GBFV.Dec}(\text{sk}, \text{GBFV.Add}(\text{ct}_1, \text{ct}_2)) = m_1 + m_2 \text{ and}$$

$$\text{GBFV.Dec}(\text{sk}, \text{GBFV.Mult}(\text{ct}_1, \text{ct}_2)) = m_1 \cdot m_2.$$

Each homomorphic operation increases the underlying noise. If we continue performing operations, the underlying noise would become too large and cause decryption failure. Bootstrapping is a homomorphic operation that significantly reduces the noise. That is, given a ciphertext $\text{ct} \in \mathcal{R}_q^2$ encrypting a message m with a noise e , the bootstrapped ciphertext $\text{GBFV.Bootstrap}(\text{ct})$ encrypts the same message m while having an underlying error e' that is significantly smaller than e . Here the smallness can be defined via a typical norm like $\|\cdot\|_2$ or $\|\cdot\|_\infty$.

² For simplicity, we do not discuss other operations like automorphism.

2.2 CKKS Basics

The discrete Fourier transform (DFT) from $\mathbb{R}[X]/(X^N + 1)$ to $\mathbb{C}^{N/2}$ is defined as

$$\text{DFT}(\alpha(X)) = (\alpha(\zeta^{5^i}))_{0 \leq i < N/2}$$

where ζ is a primitive $2N$ -th root of unity. The CKKS encoding $\text{Ecd} : \mathbb{C}^{N/2} \rightarrow \mathcal{R}$ is defined as

$$\text{CKKS.Ecd}(z) = \lfloor \Delta \cdot \text{DFT}^{-1}(z) \rfloor$$

where $\Delta > 0$ is a scaling factor. The decoding $\mathcal{R} \rightarrow \mathbb{C}^{N/2}$ is defined as its approximate inverse,

$$\text{CKKS.Dcd}(m(X)) = \frac{1}{\Delta} \cdot \text{DFT}(m(X))$$

so that $\text{Dcd} \circ \text{Ecd}$ is an approximate identity.

Encryption and Decryption. Given a message $z \in \mathbb{C}^{N/2}$, the encryption of m under the secret key $\text{sk} = (1, s) \in \mathcal{R}^2$ is defined as

$$\text{CKKS.Enc}(\text{sk}, z) = (-as + \text{CKKS.Ecd}(z) + e, a) \in \mathcal{R}_q^2$$

where a is uniformly sampled from $\mathcal{R}_q$ and e is sampled from an error distribution. A CKKS ciphertext is of the form

$$(b, a) \in \mathcal{R}_q^2$$

where $\text{pt} = \text{ct} \cdot \text{sk}$ is called as the underlying plaintext and $z = \text{CKKS.Dcd}(\text{pt})$ is called as the underlying message. Given a ciphertext $\text{ct} \in \mathcal{R}_q^2$, the decryption with respect to the secret sk is defined as

$$\text{CKKS.Dec}(\text{sk}, \text{ct}) = \text{CKKS.Dcd}(\text{ct} \cdot \text{sk}) \in \mathbb{C}^{N/2}.$$

The decryption always succeeds, but CKKS lives with error in the sense that error is always accumulated in the least significant bits.

Homomorphic Operations. As a black box, CKKS supports homomorphic addition, multiplication, rotation, and conjugation. Given two ciphertexts $\text{ct}_1, \text{ct}_2 \in \mathcal{R}_q^2$ encrypting $z_1, z_2 \in \mathbb{C}^{N/2}$, respectively, addition and multiplication denoted as

$$\text{CKKS.Add}(\text{ct}_1, \text{ct}_2) \text{ and } \text{CKKS.Mult}(\text{ct}_1, \text{ct}_2)$$

satisfy

$$\text{CKKS.Dec}(\text{sk}, \text{CKKS.Add}(\text{ct}_1, \text{ct}_2)) = z_1 + z_2 \text{ and}$$

$$\text{CKKS.Dec}(\text{sk}, \text{CKKS.Mult}(\text{ct}_1, \text{ct}_2)) = z_1 \odot z_2$$

where $\odot$ denotes the Hadamard (element-wise) multiplication. Given a ciphertext $\text{ct} \in \mathcal{R}_q^2$ encrypting $\mathbf{z} = (z_0, z_1, \dots, z_{N/2-1}) \in \mathbb{C}^{N/2}$, rotation (by index i) and conjugation denoted as

$$\text{CKKS.leftRot}_i(\text{ct}) \text{ and } \text{CKKS.Conj}(\text{ct})$$

satisfy

$$\text{CKKS.Dec}(\text{sk}, \text{CKKS.leftRot}_i(\text{ct})) = (z_i, z_{i+1}, \dots, z_{i-1}) \text{ and}$$

$$\text{CKKS.Dec}(\text{sk}, \text{CKKS.Conj}(\text{ct})) = \bar{\mathbf{z}} = (\bar{z}_0, \bar{z}_1, \dots, \bar{z}_{N/2-1}).$$

Unlike (G)BFV, CKKS allows the error to contaminate the message, so there is no need to refresh the error. However, each multiplication consumes some modulus, and we need to recover the modulus. CKKS bootstrapping is a homomorphic operation that recovers the modulus while approximately keeping the underlying message. That is, given a ciphertext $\text{ct} \in \mathcal{R}_q^2$ encrypting a message $\mathbf{z}$, the bootstrapping ciphertext $\text{CKKS.Bootstrap}(\text{ct}) \in \mathcal{R}_Q^2$ lives in a larger modulus $Q > q$ while having the underlying message $\mathbf{z}' \approx \mathbf{z}$.

2.3 Ring Packing

Ring packing [BGGJ20, CDKS21, BCK⁺23] is a homomorphic operations that packs many LWE ciphertexts into a single RLWE ciphertext.

- **Extraction:** Given a RLWE ciphertext $\text{ct} \in \mathcal{R}_Q^2$ that encrypts a plaintext $m = \sum_{i=0}^{N-1} m_i \cdot X^i \in \mathcal{R}$, we may extract N LWE ciphertexts $\text{ct}_0, \dots, \text{ct}_{N-1}$ that encrypt $m_0, m_1, \dots, m_{N-1}$, respectively. We write

$$(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{N-1}) = \text{Extract}(\text{ct}).$$

- **Ring packing:** Given $k \leq N$ LWE ciphertexts $\text{ct}_0, \dots, \text{ct}_{k-1}$, we may pack these LWE ciphertexts into a single RLWE ciphertext ct that encrypts a vector $(m_0, \dots, m_{k-1})$. We write

$$\text{ct} = \text{RingPack}(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{k-1}).$$

Here we did not specify how the the vector $(m_0, \dots, m_{k-1})$ is encoded inside the ciphertext ct . This is intentional and will be specified in each of the constructions.

2.4 Integer Computers from Discrete CKKS

Discrete CKKS [DMPS24] is built upon CKKS to instantiate exact computations. For simplicity, we focus on the message space $\mathbb{Z}_m^{N/2}$ for an integer $m > 0$.

Discrete CKKS Framework. Consider an inclusion $x \mapsto x : \mathbb{Z}_m \hookrightarrow \mathbb{C}$ (resp. $\mathbb{Z} \hookrightarrow \mathbb{C}$). A Discrete CKKS ciphertext encrypting a vector $\mathbf{z} \in \mathbb{Z}_m^{N/2}$ (resp. $\mathbb{Z}^{N/2}$) is a CKKS ciphertext that decrypts to $\mathbf{z} + \mathbf{e} \in \mathbb{C}^{N/2}$ where $\mathbf{e}$ is a small error. As long as $\|\mathbf{e}\|_\infty < 1/2$, we can round $\mathbf{z} + \mathbf{e}$ and get $\mathbf{z}$. In this regard, although Discrete CKKS is based on the approximate scheme, it is as exact as any other exact schemes like BGV/BFV/CGGI/DM. Note that the message space naturally extends to any bounded subset of $\mathbb{Z}$.

As in (G)BFV, error accumulates through computation. In order to achieve unbounded number of computations, we need to refresh the error just as in (G)BFV. This can be instantiated via a cleaning polynomial that has vanishing derivatives at $\mathbb{Z}_m$. For instance, the polynomial $h_1(x) = 3x^2 - 2x^3$ (from [CKK⁺19]) satisfies $h_1(0) = 0$, $h_1(1) = 1$, and $h'_1(0) = h'_1(1) = 0$ so it effectively cleans $\mathbb{Z}_2$. In this regard, Discrete CKKS has two bootstrappings: One for raising the modulus (i.e. the original CKKS bootstrapping) and the other to reduce the noise/error.

CKKS-style Functional Bootstrapping. One of the key ingredients of Discrete CKKS is CKKS-style functional bootstrapping [BCKS24, BKSS24, AKP25] which evaluates an arbitrary function in addition to bootstrapping. A typical syntax of a functional bootstrapping can be written as follows. Let $f : \mathbb{Z}_m \rightarrow \mathbb{Z}_m$ be an arbitrary function. Given a Discrete CKKS ciphertext ct encrypting a vector $\mathbf{z} \in \mathbb{Z}_m^{N/2}$, a CKKS-style functional bootstrapping with respect to f outputs $\text{LUT}_f(\text{ct}) \in \mathcal{R}_Q^2$ that decrypts to $f(\mathbf{z}) \in \mathbb{Z}_m^{N/2}$, where f is applied element-wise.

[KN25] observes that one can add an additional RLWE modular reduction inside CKKS-style functional bootstrapping, which gives a homomorphic modular reduction. We denote it as IntMod_m where the input Discrete CKKS ciphertext ct encrypts a vector $\mathbf{z} \in \mathbb{Z}^{N/2}$ and the output $\text{IntMod}_m(\text{ct})$ that decrypts to $[\mathbf{z}]_m \in \mathbb{Z}_m^{N/2}$. Here $[\cdot]_m$ denotes the element-wise modular reduction modulo m .

Discrete CKKS as a black box. We provide a simplified overview of Discrete CKKS. Discrete CKKS supports the following homomorphic operations:

- **Arithmetic Operations:** Given two Discrete CKKS ciphertexts ct_1, ct_2 encrypting $\mathbf{z}_1, \mathbf{z}_2 \in \mathbb{Z}^{N/2}$, respectively, we have addition and multiplication inherited from CKKS. That is, $\text{Add}(\text{ct}_1, \text{ct}_2)$ and $\text{Mult}(\text{ct}_1, \text{ct}_2)$ decrypts to $\mathbf{z}_1 + \mathbf{z}_2$ and $\mathbf{z}_1 \odot \mathbf{z}_2$, respectively.
- **Look-up Table:** Given a Discrete CKKS ciphertext ct encrypting a vector $\mathbf{z} \in \mathbb{Z}_m^{N/2}$ and a function $f : \mathbb{Z}_m \rightarrow \mathbb{Z}_m$, we can evaluate a look-up table f and output $\text{LUT}_f(\text{ct})$ that decrypts to $f(\mathbf{z})$. The cost of the look-up table is approximately as costly as CKKS bootstrapping.
- **Modular Reduction:** Given a Discrete CKKS ciphertext ct encrypting a vector $\mathbf{z} \in \mathbb{Z}^{N/2}$, the homomorphic modular reduction $\text{IntMod}_m : \mathcal{R}_q^2 \rightarrow \mathcal{R}_Q^2$ outputs $\text{IntMod}_m(\text{ct})$ that decrypts to $[\mathbf{z}]_m \in \mathbb{Z}_m^{N/2}$. The cost of the modular reduction is approximately as costly as CKKS bootstrapping.

Note that both LUT_f and IntMod_m inherently includes bootstrapping. That is, they both raise the modulus and refresh the error/noise.

Nevertheless, we consider some black box cleaning functions for later constructions. Let $\text{Clean}_b : \mathcal{R}_Q^2 \rightarrow \mathcal{R}_Q^2$ be a $\mathbb{Z}_b$ cleaning function to a Discrete CKKS ciphertext encoded either in slots or coeffs. It can either contain polynomial evaluations of vanishing derivatives [DMPS24] or functional bootstrapping that inherently has cleaning [BKSS24, AKP25], or both.

Definition 1. A cleaning function $\text{Clean}_b : \mathcal{R}_Q^2 \rightarrow \mathcal{R}_Q^2$ with a cleaning factor (B, B') satisfies the following: For any ciphertext ct whose underlying error satisfies $\|e\|_\infty < B$, the output ciphertext $\text{Clean}_b(\text{ct})$ has error satisfying $\|e'\|_\infty < B'$.

CKKS-based Integer Computers. Recent works [Kim25b, BK25, Kim25c, CPL25] build fully homomorphic encryption schemes for integer arithmetic upon the Discrete CKKS framework. We briefly describe the ones from [Kim25b] and [CPL25], as they are used in our constructions.

Let us consider representing integer to its digit form, for a radix base b . For instance, an integer $x < b^k$ can be written as

$$x = \overline{x_{k-1}x_{k-2} \cdots x_0} (b).$$

[Kim25b] stores one ciphertext per digit. That is, there is a ciphertext ct_i that stores the i -th digit x_i . To extend this to SIMD, given an integer vector $\mathbf{z} \in \mathbb{Z}^{N/2}$, the i -th ciphertext ct_i stores all the i -th digits of each component. Such an encoding, together with the Discrete CKKS framework described above, leads to having a SIMD integer computer: Each digit is managed through Discrete CKKS operations over $\mathbb{Z}_b$, and we have some cross-digit operations in addition. As a result, they achieve a homomorphic encryption scheme over $\mathbb{Z}_{b^k}^{N/2}$ that supports arithmetic operations like addition and multiplication and logical operations like comparison and bitwise operations at the same time. We mainly rely on the following two types of operations.

- **Addition and Multiplication:** Given ciphertext vectors $\text{ct} = (\text{ct}_0, \dots, \text{ct}_{k-1})$, $\text{ct}' = (\text{ct}'_0, \dots, \text{ct}'_{k-1}) \in (\mathcal{R}_q^2)^k$ encrypting integer vectors $\mathbf{x}, \mathbf{x}' \in \mathbb{Z}_{b^k}^{N/2}$, we have addition and multiplication denoted as IntCom.Add and IntCom.Mult , which evaluate vectorized addition and multiplication over $\mathbb{Z}_{b^k}$, respectively.
- **Comparison:** Given ciphertext vectors $\text{ct} = (\text{ct}_0, \dots, \text{ct}_{k-1})$, $\text{ct}' = (\text{ct}'_0, \dots, \text{ct}'_{k-1}) \in (\mathcal{R}_q^2)^k$ encrypting an integer vectors $\mathbf{x}, \mathbf{x}' \in \mathbb{Z}_{b^k}^{N/2}$, we have a comparison operation IntCom.Compare , which evaluates vectorized greater than or equal ($\geq$) operation over $\mathbb{Z}_{b^k}$.

Here, addition runs in time $O(k)$, multiplication runs in time $O(k \log k)$, and comparison runs in time $O(k)$.

[CPL25] packs multiple digits inside a single ciphertext. In particular, the digits are simply stored in the CKKS slots, with respect to a fixed permutation. We rely on the following operations.

- **Addition and Multiplication:** Given ciphertexts $\text{ct}, \text{ct}' \in \mathcal{R}_q^2$ encrypting integer vectors $\mathbf{x}, \mathbf{x}' \in \mathbb{Z}_{b^k}^{N/(4k)}$, we have addition and multiplication denoted as `ParCom.Add` and `ParCom.Mult`, which evaluate vectorized addition and multiplication over $\mathbb{Z}_{b^k}$, respectively.
- **Comparison:** Given ciphertexts $\text{ct}, \text{ct}' \in \mathcal{R}_q^2$ encrypting integer vectors $\mathbf{x}, \mathbf{x}' \in \mathbb{Z}_{b^k}^{N/(4k)}$, we have a comparison operation `ParCom.Compare`, which evaluates vectorized greater than equal $\geq$ operation over $\mathbb{Z}_{b^k}$.
- **Modular Reduction:** Given a ciphertext $\text{ct} \in \mathcal{R}_q^2$ and a target modulus p , we have a modular reduction operation `ParCom.REDCp`, which evaluates vectorized modular reduction $[\cdot]_p$ over $\mathbb{Z}_{b^k}$. This can either be Montgomery or Base reduction, and we use this as a black box.

During the computations, the radix encoding can be lazily reduced. That is, the radix representation of an integer $x \in \mathbb{Z}_{b^k}$ may encode a vector $(x_0, x_1, \dots, x_{k-1})$ where x_i could possibly be larger than b and satisfy

$$x = \sum_{i=0}^{k-1} b^i \cdot x_i \pmod{b^k}.$$

In [CPL25], they design a reduction mechanism that runs in time $O(\log k)$ and reduces the values $x_0, \dots, x_{k-1}$ into $[0, b)$ so that it becomes the unique radix representation of the integer x . For the operations we discussed above, we allow that the input to be lazily reduced (i.e., possibly exceeding b), but the output will be exactly reduced.

2.5 Conversion from GBFV to CKKS

In [Kim25a], they design a conversion from GBFV to CKKS. At a high level, a GBFV ciphertext encrypting a vector of integers modulo $p = b^k + 1$, the resulting ciphertext is a Discrete CKKS ciphertext that essentially encrypts the approximate digit decompositions of the original message. That is, let $m = (m_0, m_1, \dots, m_{N/k-1}) \in \mathbb{Z}_p^{N/k}$ be the original message then each m_i is decomposed into integers $m_{i0}, m_{i1}, \dots, m_{i(k-1)}$ such that

$$m_i = \sum_{j=0}^{k-1} b^j \cdot m_{ij} \pmod{p}$$

and $|m_{ij}| \leq 3(b+1)/2$ for each j . The output Discrete CKKS ciphertext has m_{ij} 's in each coefficient. Let `ConvGtC` and `ConvCtG` be the conversion from GBFV to CKKS and vice versa.

3 Conversion between BFV and Discrete CKKS

In this section, we describe how to instantiate a conversion from BFV to Discrete CKKS and vice versa, in particular to the encoding of the integer computer from [Kim25b]. In addition, we discuss its applications, including asymptotically faster homomorphic comparison for BFV and many other operations.

3.1 Ingredients

We briefly discuss the ingredients for our construction, which are minor modifications of the MSB bootstrapping in [KN25] and homomorphic modular reduction from [Kim25b,CPL25].

Digit Decomposition. We adapt the iterative MSB bootstrapping from [KN25] to start from coefficients encoding and end at slots encoding, and does not combine the final results into a single ciphertext but output k ciphertext pairs. This provides an efficient digit decomposition for a Discrete CKKS ciphertext encoded in the most significant bits. The detailed algorithm is provided in Algorithm 1.

Algorithm 1: IntDecomp $_b^k$

Setting: $\Delta_0 = q_0/b^k$. CoeffIntBoot_b denotes identity functional bootstrapping from coefficients encoding to slots encoding for LUT modulus b . It outputs two ciphertexts, one for the real part and the other for the imaginary part.

Input : $\text{ct} = \text{Enc}_{\text{sk}} \circ \text{Ecd}(m) \in \mathcal{R}_{q_0}^2$ a coefficient-encoded CKKS ciphertext where m is a vector of integers in $[0, b^k)$.

Output: $(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{k-1})$ encrypting the digit decomposed result of ct .

```

1  $\text{ct}' \leftarrow \text{ct}$ ;
2 for  $i \leftarrow 0$  to  $k - 1$  do
3    $\text{ct}_i = (\text{ct}_i^R, \text{ct}_i^I) \leftarrow \text{CoeffIntBoot}_b(b^{k-1-i} \cdot \text{ct}')$ ;
4   if  $i \neq k - 1$  then
5      $\text{ct}' \leftarrow \text{ct}' - b^i \cdot \text{StC}(\text{ct}_i^R + i \cdot \text{ct}_i^I)$ ;
6   end if
7 end for
8 return  $(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{k-1})$ 

```

Note that the resulting ciphertext vector can be regarded as having an encoding structure of the integer computer from [Kim25b]. Although each digit contains two ciphertexts $\text{ct}_i^R, \text{ct}_i^I$ not one, we may regard that real part and imaginary part are encoding independently to two ciphertexts $(\text{ct}_0^R, \text{ct}_1^R, \dots, \text{ct}_{k-1}^R)$ and $(\text{ct}_0^I, \text{ct}_1^I, \dots, \text{ct}_{k-1}^I)$. Homomorphic operations can be applied in parallel on both parts.

General Modular Reduction. Recall that [Kim25b] supports various operations including addition, multiplication, comparison, and bit shift: which is enough to simulate Montgomery reduction in the encrypted world. Indeed, [CPL25] instantiated the same idea in their integer computer. Instead of running the algorithm in a parallel computer, we run the algorithm in [Kim25b]. The detailed algorithm is given in Algorithm 2.

We observe that the asymptotic bottleneck of the algorithm is two homomorphic multiplications, which require $O(k \log k)$ discrete bootstrappings.

Algorithm 2: IntCom.REDC_p

Setting: $p < b^k$ is the target modulus that is coprime with the digit base b .
 $p' = p^{-1} \bmod b^k$. IntCom.MultExt denotes the extended multiplication between two k -bit integers, outputting a $2k$ -bit integer. IntCom.High takes the high part of the $\mathbb{Z}_{b^{2k}}$ integer(s).
 IntCom.CondSub($\cdot, y$) is a homomorphic encryption of the function $x \geq y ? x - y : x$.

Input : A ciphertext vector $\mathbf{ct} = (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{k-1})$ encrypting a message $m \in \mathbb{Z}_{b^k}^{N/2}$ according to the encoding from [Kim25b].

Output: A ciphertext vector $\mathbf{ct}' = (\text{ct}'_0, \text{ct}'_1, \dots, \text{ct}'_{k-1})$ encrypting $[m]_p$.

- 1 $\mathbf{ct}' \leftarrow \text{IntCom.Mult}(\mathbf{ct}, p')$;
- 2 $\mathbf{ct}' \leftarrow \text{IntCom.High}(\mathbf{ct} + \text{IntCom.MultExt}(\mathbf{ct}', p))$;
- 3 $\mathbf{ct}' \leftarrow \text{IntCom.CondSub}(\mathbf{ct}', p)$;
- 4 **return** $(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{k-1})$

3.2 Conversions

The MSB bootstrapping from [KN25] iterates the identity functional bootstrapping to bootstrap a coefficient-encoded Discrete CKKS ciphertext whose message is encoded in the most significant bits. Since this encoding is exactly the same as the BFV encoding structure, this automatically gives an efficient conversion from BFV to Discrete CKKS for a radix power modulus $p = b^k$. The cost of this procedure is roughly k times the $\mathbb{Z}_b$ functional bootstrapping.

For general BFV modulus, this method needs to directly achieve modulus p with one iteration. Since the cost of modulo p functional bootstrapping is $O(p)$, achieving NTT prime like $p = 2^{16} + 1$ is very expensive. We propose a novel MSB bootstrapping that can handle non-radix-power moduli, using the integer computer from [Kim25b].

Consider a coefficient-encoded BFV ciphertext $\text{ct} \in \mathcal{R}_q^2$ with plaintext modulus p (not necessarily a power-of-radix). It satisfies

$$\text{ct} \cdot \text{sk} = \lfloor (q/p) \cdot m \rfloor + e$$

for some message $m \in \mathcal{R}_p$ and noise $e \in \mathcal{R}$. As we raise the modulus (i.e. embedding into $\mathcal{R}$), we may write

$$\text{ModRaise}(\text{ct}) \cdot \text{sk} = \lfloor (q/p) \cdot m \rfloor + e + q \cdot I$$

for some small integral I . We can interpret this ciphertext as a Discrete CKKS ciphertext encrypting $m + pI$:

$$\text{ModRaise}(\text{ct}) \cdot \text{sk} = \lfloor \Delta \cdot (m + pI) \rfloor + e$$

where $\Delta = q/p$ is the scaling factor. We know that $m + pI$ follows the Irwin-Hall distribution and that $\|m + pI\|_\infty < K$ for some integer K with overwhelming probability. We add pK to all the coefficients and get $\text{ModRaise}(\text{ct}) + pK$. We

choose k to satisfy $b^k > 2K + 1$ so that each coefficient of $m + pI + pK$ belongs to $[0, b^k)$ with overwhelming probability. By taking modulo $q' = \Delta \cdot b^k$, the ciphertext $[\text{ModRaise}(\text{ct}) + pK]_{q'}$ satisfies

$$[\text{ModRaise}(\text{ct}) + pK]_{q'} \cdot \text{sk} = \left\lfloor q' \cdot \frac{m + pI + pK}{b^k} \right\rfloor + e$$

which is a valid input for the MSB bootstrapping in [KN25]. The digit-decomposition version of their MSB bootstrapping, which we denoted as IntDecomp_b^k , provides a conversion to the encoding in [Kim25b]. That is, now we have ciphertexts $\text{ct}_0 = (\text{ct}_0^R, \text{ct}_0^I), \dots, \text{ct}_{k-1} = (\text{ct}_{k-1}^R, \text{ct}_{k-1}^I) \in (\mathcal{R}_Q^2)^2$ such that

$$\sum_{i=0}^{k-1} b^i \cdot \left\lfloor \frac{1}{\Delta} \cdot \text{CKKS.Dec}(\text{ct}_i^R, \text{ct}_i^I) \right\rfloor = m + pI + pK$$

up to some reordering after decryption.

Next, we homomorphically evaluate modulo p operation and get $\text{ct}'_0, \dots, \text{ct}'_{k-1}$ which decrypts to m instead of $m + pI + pK$. For general modular reduction, we may rely on the modular reduction IntCom.REDC , but other methods (e.g. base reduction) can also be used. To summarize, we start from a BFV ciphertext encrypting m and result in a [Kim25b] ciphertext encrypting m , which gives a conversion from BFV to Discrete CKKS. The full algorithm is given in Algorithm 3.

Algorithm 3: ConvBtD: Conversion from BFV to Discrete CKKS

Setting: $q' = \Delta \cdot b^k$ where $m + pI + pK \in [0, b^k)$ with overwhelming probability. Here K is the upper bound for the Irwin-Hall distribution.

Input : $\text{ct} = \text{BFV.Enc}(m) \in \mathcal{R}_q^2$ a BFV ciphertext encrypting a message $m \in \mathcal{R}_p$.

Output: $\text{ct}_{\text{out}} \in ((\mathcal{R}_Q^2)^2)^k$ encrypting a message m with respect to the encoding in [Kim25b].

- 1 $\text{ct}' \leftarrow [\text{ModRaise}(\text{ct})]_{q'} + pK$;
 - 2 $\text{ct} \leftarrow \text{IntDecomp}_b^k(\text{ct}')$;
 - 3 $\text{ct}_{\text{out}} \leftarrow \text{IntCom.REDC}_p(\text{ct})$;
 - 4 **return** ct_{out}
-

Theorem 1. *If the input BFV ciphertext ct has sufficiently small noise, the BFV to Discrete CKKS conversion in Algorithm 3 is correct. The conversion involves $O(k \log k)$ discrete bootstrappings.*

Proof. Since ct is a coefficient-encoded BFV ciphertext, it can be understood as a Discrete CKKS ciphertext whose message is encoded in the most significant bits.

If ct has a sufficiently small underlying noise, it can be regarded as a valid input for the MSB bootstrapping of [KN25]. After modulus raising, the normalization factor pK ensures that we get an unsigned value $m + p(I + K)$ which modulus reduces to m after taking modulo p . The normalization ciphertext ct' is decomposed using the digit-decomposed style MSB bootstrapping `IntDecomp` and becomes a valid [Kim25b] ciphertext. The correctness of the MSB bootstrapping ensures that ct has sufficiently small noise to allow further computation in the [Kim25b] framework. Finally, the homomorphic modular reduction by `IntCom.REDC` reduces $m + p(I + K)$ to m , achieving the right output ciphertext as desired.

We now check the asymptotic computation complexity of the algorithm. First, modulus raising and addition of pK is $O(1)$, as are negligible. `IntDecomp` _{b^k} requires k executions of modulo b discrete bootstrappings, so it is $O(k)$. Finally, the Montgomery reduction `REDC` _{p} has complexity $O(k \log k)$. Therefore, we get

$$O(k \log k) + O(k) + O(1) = O(k \log k).$$

This finishes the proof. $\square$

We discuss a conversion from [Kim25b] to BFV, which is almost automatic. The key observation is that Discrete CKKS and BFV are exactly the same when they are encrypted in the coefficients. Therefore, a simple recombination in the Discrete CKKS format should provide a conversion from [Kim25b] to BFV. One caveat is that the combined ciphertext is noisier than usual, so we need sufficient cleaning to compensate this. The detailed algorithm is described in Algorithm 4.

Algorithm 4: ConvDtB: Conversion from Discrete CKKS to BFV

Setting: Δ is a Discrete CKKS scaling factor.

Input : $\text{ct} \in ((\mathcal{R}_Q^2)^2)^k$ a Discrete CKKS ciphertext vector encrypting $m \in \mathbb{Z}_{b^k}^N$ according to the encoding in [Kim25b].

Output: $\text{ct}_{\text{out}} \in \mathcal{R}_q^2$ encrypting a message m in BFV coefficients.

```

1  $\text{ct}_{\text{out}} \leftarrow \text{ct}_0$  ;
2 for  $i \leftarrow 1$  to  $k - 1$  do
3    $\text{ct}_{\text{out}} \leftarrow \text{ct}_{\text{out}} + b^i \cdot \text{Clean}_b(\text{ct}_i)$ ;
4 end for
5  $\text{ct}_{\text{out}} \leftarrow \text{StC}(\text{ct}_{\text{out}}^R + i \cdot \text{ct}_{\text{out}}^I)$ ;
6 return  $\text{ct}_{\text{out}}$ 
```

Theorem 2. *If there exists $B > 0$ and $0 < B' \ll \frac{1}{kp}$ such that each ct_i has underlying error e_i satisfying $\|e_i\|_\infty < B_i$ and Clean_b for ct_i has a cleaning factor $(B, B' \cdot b^{-i})$, then the conversion in Algorithm 4 is correct.*

Proof. First, it is clear that the message that ct_{out} is

$$\sum_{i=0}^{k-1} b^i \cdot m_i = m$$

where m_i is the underlying message of ct_i for each i as a Discrete CKKS ciphertext. Thus, it remains to show that the underlying error is sufficiently small. Since ct_i has underlying error e_i satisfying $\|e_i\|_\infty < B$, it is a valid input for Clean_b according to the cleaning factor $(B, B' \cdot b^{-i})$. Hence, the output ciphertext $\text{Clean}_b(\text{ct}_i)$ has underlying error e'_i satisfying $\|e'_i\|_\infty < B' \cdot b^{-i}$. Since ct_{out} is defined as

$$\text{ct}_{\text{out}} = \sum_{i=0}^{k-1} b^i \cdot \text{Clean}_b(\text{ct}_i),$$

ct_{out} has an underlying error e' that satisfies

$$\|e'\|_\infty \leq \sum_{i=0}^{k-1} b^i \cdot (B' \cdot b^{-i}) = k \cdot B' \ll 1/p.$$

We assume that StC is sufficiently precise and does not add significant error. In that case, ct_{out} is a valid BFV ciphertext encrypting the right message. $\square$

Because of the correctness of the conversion algorithms, it naturally follows that the composition of two conversions is an identity. This is a straightforward result from the correctness theorems Theorems 1 and 2 so we omit the proof.

Corollary 1. *For a sufficiently precise conversions ConvBtD and ConvDtB , we have that*

$$\text{ConvDtB} \circ \text{ConvBtD}$$

is a homomorphic identity function for BFV (possibly having different noise terms), given that the input ciphertext has sufficiently small noise.

3.3 Applications to BFV

We discuss the applications of the conversion in the context of BFV. We first observe that the bit and digit decompositions are immediate, and then discuss logical operations like comparison and bit-wise operations.

Decompositions. A natural observation is that the conversion is inherently a digit decomposition: The resulting format is encoded according to [Kim25b], which stores the integers as digit decomposed format of base b . Therefore, conversion from BFV to [Kim25b] and converting back the individual ciphertext (instead of deriving the recombination) gives a digit decomposition. In addition to this, these digits can further be decomposed into bits as in [KN25].

Comparison. By using the conversion from BFV to Discrete CKKS as a sub-routine, we describe how we instantiate a BFV comparison. Since we have a conversion from BFV to [Kim25b], we can use the integer computer operations in [Kim25b] to compute comparison. After computing comparison, we get a Discrete CKKS ciphertext encrypting comparison bits (1 if $a \geq b$ and 0 if $a < b$).

Algorithm 5: CompBFV: BFV Comparison from Discrete CKKS

Setting: Δ is a Discrete CKKS scaling factor.
Input : $\text{ct}_i = \text{BFV.Enc}(m_i) \in \mathcal{R}_q^2$ BFV ciphertexts for $i = 1, 2$.
Output: $\text{ct}_{\text{out}} \in \mathcal{R}_q^2$ encrypting a comparison bit vector.
1 $\text{ct}_i \leftarrow \text{ConvBtD}(\text{ct}_i)$ for $i = 1, 2$;
2 $\text{ct}' \leftarrow \text{IntCom.Compare}(\text{ct}_1, \text{ct}_2)$;
3 $\text{ct}_{\text{out}} \leftarrow [(q/\Delta) \cdot \text{StC} \circ \text{Clean}(\text{ct}')]_q$;
4 **return** ct_{out}

This comparison bit ciphertext can be converted back to BFV with proper cleaning. The detailed algorithm is described in Algorithm 5.

We provide an asymptotic analysis (in the plaintext modulus p) of the suggested BFV comparison. As observed before, ConvBtD runs in time $O(k \log k) = O(\log p \log \log p)$. The homomorphic comparison from [Kim25b] runs in time $O(k) = O(\log p)$. For cleaning, it depends on the size of the target noise. If we treat it as a constant, then it would be $O(1)$. If we aim to obtain some multiplicative depths t after comparison, the resulting precision needs to be $t \log p$. To clean this amount, we need $O(\log \log p)$ iterations of the base cleaning (e.g. $h_1(x) = 3x^2 - 2x^3$). As a result, the overall asymptotic complexity is $O(\log p \log \log p)$.

Bit-wise Operations. Since bit decomposition outputs encryptions of individual bits, we may run BFV bit operations on each bit to support bit-wise operations. Recall that bit operations like and/or/nand consume one multiplicative level and is very efficient. The overall asymptotic complexity is $O(k) = O(\log p \log \log p)$ as the conversion dominates.

4 Conversion between GBFV and Discrete CKKS

In this section, we extend our construction to GBFV, relying on the integer computer in [CPL25]. Our conversion can be understood as making the conversion in [Kim25a] exact.

4.1 Exact Conversion

We first propose an exact conversion from GBFV to Discrete CKKS, relying on the modular reduction (by $p = b^k + 1$) in [CPL25]. That is, instead of getting an approximate decomposition as in [Kim25a], the resulting ciphertext encrypts the unique digit decomposition of the underlying message.

The key idea is that a GBFV ciphertext multiplied by the plaintext modulus t can be understood as a Discrete CKKS ciphertext encrypting an approximate digit decomposition in coefficients, and the modular reduction in [CPL25] exactly reduces it through modular reduction and exact digit decomposition.

The asymptotic time complexity depends on the modular reduction for [CPL25] which is $O(\log k) = O(\log \log p)$. The detailed algorithm is given in Algorithm 6.

Algorithm 6: ConvGtD: Conversion from GBFV to [CPL25]

Setting: π is a permutation that orders LWE ciphertexts in a way that the packed ciphertext follows the encoding structure of [CPL25].

RingPack* is a variation of **RingPack** that outputs eight real-value-encoded ciphertexts with $2k$ zeros in the MSB.

Input : $\mathbf{ct} = \text{GBFV.Enc}(m)$ for $m \in \mathbb{Z}_p^{N/k}$.

Output: $\mathbf{ct}_{\text{out}} \in (\mathcal{R}_Q^2)^8$ a tuple of [CPL25] ciphertexts encrypting m in their slots.

```

1  $\mathbf{ct}' \leftarrow t \cdot \mathbf{ct}$ ;
2  $(\mathbf{ct}_0, \mathbf{ct}_1, \dots, \mathbf{ct}_{N-1}) \leftarrow \pi \circ \text{Extract}(\mathbf{ct}')$ ;
3  $\mathbf{ct}_{\text{out}} \leftarrow \text{RingPack}^*(\mathbf{ct}_0, \dots, \mathbf{ct}_{N-1})$ ;
4  $\mathbf{ct}_{\text{out}} \leftarrow \text{ParCom.REDC}_p(\mathbf{ct}_{\text{out}})$ ;
5 return  $\mathbf{ct}_{\text{out}}$ 

```

Theorem 3. *The conversion algorithm in Algorithm 6 is correct. The algorithm runs in time $O(\log k) = O(\log \log p)$.*

Proof. We first check correctness. After multiplying by t , we have a Discrete CKKS ciphertext containing approximate digit decomposition in their coefficients. To be explicit, let $m = (m_0, m_1, \dots, m_{N/k-1}) \in \mathbb{Z}_p^{N/k}$. Each m_i is decomposed into integers $m_{i0}, m_{i1}, \dots, m_{i(k-1)}$ such that

$$m_i = \sum_{j=0}^{k-1} b^j \cdot m_{ij} \pmod{p}$$

and $|m_{ij}| \leq (b+1) \cdot K$ for each j . The output Discrete CKKS ciphertext has m_{ij} 's in each coefficient. Since m_{ij} 's are small, we have

$$m_i + p \cdot J_i = \sum_{j=0}^{k-1} b^j \cdot m_{ij}$$

where $|J_i| \leq K \cdot (1 + 1/b)$. Therefore, if we put m_{ij} 's in the right position in CKKS slots, then it leads to encryption of $m_i + p \cdot J_i$'s according to [CPL25]. We choose k to be the smallest positive integer that satisfies $b^k > 2 \cdot p \cdot K \cdot (1 + 1/b)$ and set the digit length parameter of [CPL25] to $2k$, having $k = O(\log p)$. Since [CPL25] can start from any low-norm radix encoding of an integer, the existence of a permutation π that reorders the values with respect to the encoding structure is guaranteed. Finally, the modular reduction by p gives the exact digit decomposition of m .

In terms of computational complexity, we check the cost of individual steps. Multiplication by t , extraction, permutation, and ring packing runs in time $O(1)$. The only non-constant time is the modular reduction, which takes $O(\log k) = O(\log \log p)$ time. $\square$

As described in [Kim25a, Theorem 8], the CKKS-to-GBFV conversion (denoted as **ConvCtG**) can be instantiated with a modulus switching. We can combine this conversion together with a reordering from the encoding in [CPL25] to the encoding in [Kim25a] to get the other direction. The detailed algorithm is given in Algorithm 7.

Algorithm 7: ConvDtG: Conversion from [CPL25] to GBFV

Setting: ψ is a permutation that orders LWE ciphertexts in a way that the packed ciphertext follows the encoding structure of GBFV.
BaseRingPack is a variant of ring packing that sends coefficients to coefficients, which is even cheaper than **RingPack** as observed in [BCK⁺23].

Input : $\mathbf{ct} \in (\mathcal{R}_Q^2)^8$ a tuple of [CPL25] ciphertexts encrypting $m \in \mathbb{Z}_p^{N/k} \subseteq \mathbb{Z}_{b^{2k}}^{N/k}$ in their slots.

Output: $\mathbf{ct}_{\text{out}} \in \mathcal{R}_q^2$ a GBFV ciphertext encrypting m .

```

1  $\mathbf{ct}' \leftarrow \text{StC}(\mathbf{ct});$ 
2  $(\mathbf{ct}_0, \mathbf{ct}_1, \dots, \mathbf{ct}_{N-1}) \leftarrow \psi \circ (\text{Extract}(\mathbf{ct}'));$ 
3  $\mathbf{ct}_{\text{out}} \leftarrow \text{ConvCtG} \circ \text{BaseRingPack}(\mathbf{ct}_0, \mathbf{ct}_1, \dots, \mathbf{ct}_{N-1});$ 
4 return  $\mathbf{ct}_{\text{out}}$ 

```

Theorem 4. *The conversion algorithm in Algorithm 7 is correct. The algorithm runs in time $O(1)$.*

Proof. We first check correctness. **StC** on eight ciphertexts moves the digits from slots to coefficients. The extraction would extract the digits and output a list of LWE ciphertexts each encrypting a digit. There exists a permutation ψ that permutes these LWE ciphertexts to the correct ordering to pass the CKKS-to-GBFV conversion **ConvCtG**. Since the input of the conversion contains digits in their coefficients, the ring packing algorithm should output slots-encoded CKKS ciphertext which **BaseRingPack** does. Finally, **ConvCtG** algorithm outputs a GBFV ciphertext encrypting the same message m . The correctness is checked by the individual components.

In terms of running time, all the algorithms run independent of the digit precision k . Therefore, the time complexity is $O(1)$. $\square$

4.2 Applications

We discuss the applications of the conversion to GBFV. As in BFV, the bit and digit decompositions are immediate, and logical operations also directly follow.

Decompositions. By definition, the conversion is inherently a digit decomposition with radix base b . The resulting ciphertext encodes each digit in slot of CKKS ciphertext. Therefore, converting from GBFV to CKKS and converting back individual slots automatically provides digit decomposition for GBFV. The digits can further be decomposed into bits as in [KN25]. Since the encoding of [CPL25] stores different digits in a single ciphertext, this can be done in SIMD but with different ordering than the standard ordering. If needed, one can add a simple reordering process, which can be instantiated with masking.

Logical Operations. Once we convert two GBFV ciphertexts into [CPL25] ciphertexts, we can use their comparison operation that runs in times $O(\log k) = O(\log \log p)$. After comparison, we can convert back to GBFV using the other way conversion. The detailed algorithm is written in Algorithm 8. The computational complexity of the entire algorithm is still $O(\log \log p)$.

Algorithm 8: CompGBFV: GBFV Comparison from Discrete CKKS

Setting: Δ is a Discrete CKKS scaling factor.
Input : $\text{ct}_i = \text{GBFV.Enc}(m_i) \in \mathcal{R}_q^2$ GBFV ciphertexts for $i = 1, 2$.
Output: $\text{ct}_{\text{out}} \in \mathcal{R}_q^2$ encrypting a comparison bit vector.

- 1 $\text{ct}_i \leftarrow \text{ConvGtD}(\text{ct}_i)$ for $i = 1, 2$;
- 2 $\text{ct} \leftarrow \text{ParCom.Compare}(\text{ct}_1, \text{ct}_2)$;
- 3 $\text{ct}_{\text{out}} \leftarrow \text{ConvDtG}(\text{ct})$;
- 4 **return** ct_{out}

Bit-wise Operations. After bit decomposition mentioned above, we may run GBFV bit operations on each bit to support bit-wise operations. However, this runs in time $O(\log p)$ which is not desirable. Instead, we may run comparison in the Discrete CKKS side, by running bit-wise operations on individual slots that contain bits. After bit decomposition, we have $\log_2 b = O(1)$ ciphertexts, so the overall algorithm containing the homomorphic bit decomposition still runs in time $O(\log \log p)$.

Leveled Real Number Arithmetic. Since GBFV can support up to extremely high precision, one may consider using GBFV to support fixed-point style real number arithmetic. That is, we may start with a scaled-and-rounded real number $m = \lfloor \Delta \cdot z \rfloor \in \mathbb{Z}_p$ for a large modulus p . After a multiplication, the scaling factor grows to Δ^2 . After d multiplications, the scaling factor grows to Δ^{2^d} . In BGV/BFV/CKKS, such an exponential growth is prohibitive, and this is exactly why CKKS rescales (by Δ) every time they multiply. However, in GBFV, the modulus can be as large as $\approx b^N$ which can support a few multiplications even with exponential scaling factor growth. For instance, if $p \approx \Delta^{2^d}$ for some d , then we can support roughly up to d multiplicative depth.

The problem is that this scheme is only somewhat homomorphic in the sense that we cannot reduce the scaling factor back to Δ . In other words, we need a some form of bootstrapping that not only reduces the GBFV error but also decreases the scaling factor. Our conversion directly provides this bootstrapping. That is, one starts with a very large integer, and remove almost all the data except Δ amount of data in the MSB. This operation can be instantiated with first converting a GBFV ciphertext into a discrete CKKS ciphertext according to the encoding in [CPL25], remove all but a few digits in the MSB using CKKS masking (i.e., multiplying by a vector consisting of 0's and 1's), and converting back to GBFV.

Compared to CKKS, the latency of the proposed method is comparable because we use one RLWE multiplication per each multiplication. The amortized latency, however, is significantly worse than CKKS as we sacrifice the number of slots to store enormously large integers. The biggest strength of our construction is that this fixed-point style arithmetic is now *exact* and depends only on the underlying message. Recall that CKKS requires noise flooding in many security notions, including IND-CPA-D [LM21] and Threshold FHE [BGG⁺18]. Since they cannot take the repeated bootstrapping path [DS16] and require very high precision. Our approach provides an alternative option that is exact and much more efficient than other alternatives.

5 Exploiting sparse packing for better efficiency

In prior sections, we were able to achieve asymptotically faster logical operations for BFV and GBFV. In particular, BFV comparison complexity decreased from $O(p)$ to $O(\log p \log \log p)$ and the GBFV comparison complexity decreased from $O(p)$ to $O(\log \log p)$. A natural question is whether we can do better, especially for BFV which has worse asymptotic complexity. In this section, we discuss how to achieve better complexity in the case of sparsely packed ciphertexts.

5.1 Faster Logical Operations for Sparse BFV

For Sparse BFV, we mean by BFV not fully using the RLWE coefficients, and the underlying plaintext $m \in \mathbb{Z}_p[X]/(X^N + 1)$ lives in a subring $\mathbb{Z}_p[X^k]/(X^N + 1)$ for some power-of-two k . In other words, all the coefficients other than degree multiple of k are zero. Here we consider the case where $p \cdot (2K) \leq b^k$ for some small base b , so that later we can use the framework from [CPL25].

We now describe the protocol. We start from a coefficients-encoded BFV ciphertext $\text{ct} \in \mathcal{R}_q^2$ that encrypts $m \in \mathbb{Z}_p[X^k]/(X^N + 1)$.

1. **Modulus switching:** Choose a modulus $q' = p \cdot r \approx q$ and modulus-switch ct to that modulus. Then modraise the ciphertext. It becomes a valid BFV ciphertext $\text{ct}' \in \mathcal{R}^2$ encrypting $m + p \cdot I$ for some small I such that $0 \leq I < 2K$. We may write

$$\text{ct}' \cdot \text{sk} = r \cdot (m + pI) + e$$

for some small error e . Let $m + p \cdot I = \sum_{i=0}^{k-1} b^i \cdot m_i$ be the coefficient-wise digit decomposition of $m + p \cdot I$ base b .

2. **Decomposition:** We define ct_0 as $\text{ModRaise}([\text{ct}]_{b \cdot r}) \in \mathcal{R}^2$. This can be regarded as a ciphertext that encrypts the lowest digit but with two different noises: one coming from the LSB, and the other which is a small multiple of $b \cdot r$. As a result, we have

$$\text{ct}_0 \cdot \text{sk} = r \cdot (m_0 + b \cdot I_0) + e$$

for some small integral polynomial I_0 and error e which is the same as the error in ct' . Next, we subtract this ciphertext from ct' and divide by b , getting

$$\frac{\text{ct}' - \text{ct}_0}{b} \cdot \text{sk} = r \cdot \left(\left(\sum_{i=1}^{k-1} b^{i-1} \cdot m_i \right) - I_0 \right).$$

Let $\text{ct}_1 = \text{Modraise}([\text{ct}' - \text{ct}_0]/b)_{b \cdot r}$ which satisfies

$$\text{ct}_1 \cdot \text{sk} = r \cdot (m_1 - I_0 + b \cdot I_1)$$

for some small integral polynomial I_1 . We may repeat this and get

$$\text{ct}_i \cdot \text{sk} = r \cdot (m_i - I_{i-1} + b \cdot I_i)$$

for $0 \leq i < k$. Let $m'_i = m_i - I_{i-1} + b \cdot I_i$ for each i . ct_i is a discrete CKKS encryption of m'_i with scaling factor r . Furthermore, m'_i satisfies

$$\left[\sum_{i=0}^{k-1} b^i \cdot m'_i \right]_{b^k} = m + p \cdot I$$

and is an approximate decomposition of $m + p \cdot I$.

3. **Repacking:** We first extract

$$(\text{ct}_{i0}, \text{ct}_{i1}, \dots, \text{ct}_{i(N/k-1)}) = \text{Extract}(\text{ct}_i)$$

for each i . Since ct_i is sparse, we extract only the coefficients whose degree is multiple of k and therefore having N/k LWE ciphertexts instead of N . As a total, we have $(N/k) \cdot k = N$ LWE ciphertexts. We permute these ciphertexts and ring pack so that it matches the encoding of [CPL25]. Since each [CPL25] ciphertext has $N/(8k)$ slots for modulus b^{2k} , we need eight many CKKS ciphertexts. Let $\mathbf{ct}$ be the tuple of them.

4. **Modular Reduction:** Run ParCom.REDC_p on $\mathbf{ct}$ to modular reduce by p and also obtain exact digit decomposition.

Once we get to this point, we can use the integer computer arithmetic in [CPL25] to support any kinds of logical operations efficiently. A natural question is to how to go back to the original BFV format. For this, we can extract each slot and apply the digit decomposition formula to recombine digits back into large integer format.

Efficiency Analysis. The dominant component of the conversion from sparse BFV to [CPL25] is the modular reduction in step 4. It takes $O(\log k) = O(\log \log p)$ many bootstrappings. Among the remaining computations, the decomposition step indeed has asymptotic complexity $O(k)$ because one needs to extract $\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{k-1}$. However, this operation is merely reading relevant data from ct' , and is substantially cheaper than homomorphic operations like homomorphic multiplication and bootstrapping. The other operations run in time $O(1)$. Therefore, we can analyze the asymptotic complexity of the overall algorithm as roughly $O(\log k) = O(\log \log p)$, with a small caveat that the decomposition step is technically $O(k)$.

5.2 Faster Comparison for CKKS

State-of-the-art CKKS comparison methods [CKK20, LLNK22] claim optimal asymptotic complexity of $O(k)$ for compare k -bit precise (real) numbers. This is indeed true if we compute comparison within a single slot, achieving full SIMD parallelism. As in the case of BFV, we decrease the asymptotic cost when the packing is sparse.

We start from a coefficients-encoded CKKS ciphertext $\text{ct} \in \mathcal{R}_q^2$ that encrypts $m \in \mathbb{Z}[X^k]/(X^N + 1)$ such that $0 \leq m < b^k$. That is, we have

$$[\text{ct} \cdot \text{sk}]_q = m.$$

Note that any high precision CKKS ciphertext can be written in this form. If the scaling factor is significantly larger than the target precision, one may use the quantization framework in [KN25] to fit in the suggested format. Let $m = \sum_{i=0}^{k-1} b^i \cdot m_i$ be the base b digit decomposition of m .

1. **Decomposition:** First let $\text{ct}_0 = [\text{ct}]_b$. Then it satisfies

$$\text{ct}_0 \cdot \text{sk} = m_0 + b \cdot I_0$$

for some small integral polynomial I_0 . Next, define $\text{ct}_1 = [(\text{ct} - \text{ct}_0)/b]_b$ which satisfies

$$\text{ct}_1 \cdot \text{sk} = (m_1 - I_0) + b \cdot I_1$$

for some small integral I_1 . We may repeat this and get ct_i such that

$$\text{ct}_i \cdot \text{sk} = (m_i - I_{i-1}) + b \cdot I_i$$

for $0 \leq i < k$. Let $m'_i = m_i - I_{i-1}$ for each i . Then we have

$$m = \sum_{i=0}^{k-1} b^i \cdot m'_i \pmod{b^k}$$

so that m'_i 's are the valid approximate digit decomposition of m .

2. Repacking: We extract

$$(\mathbf{ct}_{i0}, \mathbf{ct}_{i1}, \dots, \mathbf{ct}_{i(N/k-1)}) = \text{Extract}(\mathbf{ct}_i)$$

for each i . Since each $\mathbf{ct}_i$ is sparse, we extract only the coefficients whose degree is multiple of k and therefore having N/k LWE ciphertexts instead of N . As a total, we have N LWE ciphertexts. We permute these ciphertexts and ring pack so that it matches the encoding of [CPL25]. Since each [CPL25] ciphertext has $N/(4k)$ slots for modulus $\mathbb{Z}_{b^k}$, we need four CKKS ciphertexts. Let $\mathbf{ct}$ be the tuple of them.

3. **Comparison.** Given two decomposed ciphertext vectors $\mathbf{ct}, \mathbf{ct}'$, we run $\text{ParCom.Compare}(\mathbf{ct}, \mathbf{ct}')$ and output the comparison result. After some reordering, the resulting ciphertext can be represented as a sparsely packed Discrete CKKS ciphertext.

Efficiency Analysis. The dominant component of the new comparison algorithm is the homomorphic comparison in step 3. It takes $O(\log k)$ many bootstrappings. Among the remaining operations, the decomposition step indeed has asymptotic complexity $O(k)$ as in the previous section. However, this operation is merely reading data from $\mathbf{ct}'$ and is significantly cheaper than homomorphic operations like multiplication and bootstrapping. The rest of the operations run in time $O(1)$. Therefore, we can analyze the asymptotic complexity of the overall algorithm as roughly $O(\log k)$, with a small caveat on the decomposition complexity which is technically $O(k)$.

Acknowledgement

This work was partially supported by a grant from UBRI, and the author was supported by the Stanford Graduate Fellowship. We used ChatGPT for minor wording edits.

References

- [AKP25] Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General functional bootstrapping using CKKS. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 304–337. Springer, Cham, August 2025.
- [BCK⁺23] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, Jai Hyun Park, and Damien Stehlé. HERMES: Efficient ring packing using MLWE ciphertexts and application to transciphering. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 37–69. Springer, Cham, August 2023.
- [BCKS24] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. Bootstrapping bits with CKKS. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 94–123. Springer, Cham, May 2024.

- [BGG⁺18] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, August 2018.
- [BGGJ20] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. Chimera: Combining ring-lwe-based fully homomorphic encryption schemes. *Journal of Mathematical Cryptology*, 14(1):316–338, 2020.
- [BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012*, pages 309–325. ACM, January 2012.
- [BK25] Dan Boneh and Jaehyung Kim. Homomorphic encryption for large integers from nested residue number systems. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 338–370. Springer, Cham, August 2025.
- [BK26] Dan Boneh and Jaehyung Kim. A universal blinder: One-round blind signatures from FHE. Cryptology ePrint Archive, Paper 2026/574, 2026.
- [BKSS24] Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Bootstrapping small integers with CKKS. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 330–360. Springer, Singapore, December 2024.
- [Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 868–886. Springer, Berlin, Heidelberg, August 2012.
- [CDKS21] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient homomorphic conversion between (ring) LWE ciphertexts. In Kazue Sako and Nils Ole Tippenhauer, editors, *ACNS 2021, Part I*, volume 12726 of *LNCS*, pages 460–479. Springer, Cham, June 2021.
- [CGGI16] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part I*, volume 10031 of *LNCS*, pages 3–33. Springer, Berlin, Heidelberg, December 2016.
- [CHM⁺25] Hyunho Cha, Intak Hwang, Seonhong Min, Jinyeong Seo, and Yongsoo Song. MatriGear: Accelerating authenticated matrix triple generation with scalable prime fields via optimized HE packing. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 2453–2471. IEEE Computer Society Press, May 2025.
- [CKK⁺19] Jung Hee Cheon, Dongwoo Kim, Duhyeong Kim, Hun-Hee Lee, and Kee-woo Lee. Numerical method for comparison on homomorphically encrypted numbers. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part II*, volume 11922 of *LNCS*, pages 415–445. Springer, Cham, December 2019.
- [CKK20] Jung Hee Cheon, Dongwoo Kim, and Duhyeong Kim. Efficient homomorphic comparison methods with optimal complexity. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part II*, volume 12492 of *LNCS*, pages 221–256. Springer, Cham, December 2020.
- [CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi

- and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 409–437. Springer, Cham, December 2017.
- [CLPX18] Hao Chen, Kim Laine, Rachel Player, and Yuhou Xia. High-precision arithmetic in homomorphic encryption. In Nigel P. Smart, editor, *CT-RSA 2018*, volume 10808 of *LNCS*, pages 116–136. Springer, Cham, April 2018.
- [CLR17] Hao Chen, Kim Laine, and Peter Rindal. Fast private set intersection from homomorphic encryption. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1243–1255. ACM Press, October / November 2017.
- [CPL25] Gyeongwon Cha, Dongjin Park, and Joon-Woo Lee. Improved radix-based approximate homomorphic encryption for large integers via lightweight bootstrapped digit carry. *Cryptology ePrint Archive*, Report 2025/1740, 2025.
- [DKL⁺18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR TCHES*, 2018(1):238–268, 2018.
- [DM15] Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 617–640. Springer, Berlin, Heidelberg, April 2015.
- [DMPS24] Nir Drucker, Guy Moshkovich, Tomer Pelleg, and Hayim Shaul. BLEACH: Cleaning errors in discrete computations over CKKS. *Journal of Cryptology*, 37(1):3, January 2024.
- [DS16] Léo Ducas and Damien Stehlé. Sanitization of FHE ciphertexts. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part I*, volume 9665 of *LNCS*, pages 294–310. Springer, Berlin, Heidelberg, May 2016.
- [FV12] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, Report 2012/144, 2012.
- [Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st ACM STOC*, pages 169–178. ACM Press, May / June 2009.
- [GV25] Robin Geelen and Frederik Vercauteren. Fully homomorphic encryption for cyclotomic prime moduli. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 366–397. Springer, Cham, May 2025.
- [IZ21] Ilia Iliashenko and Vincent Zucca. Faster homomorphic comparison operations for BGV and BFV. *PoPETs*, 2021(3):246–264, July 2021.
- [jLHH⁺21] Wen jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. PEGASUS: Bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *2021 IEEE Symposium on Security and Privacy*, pages 1057–1073. IEEE Computer Society Press, May 2021.
- [Kim25a] Jaehyung Kim. Bootstrapping GBFV with CKKS. *Cryptology ePrint Archive*, Report 2025/888, 2025.
- [Kim25b] Jaehyung Kim. Efficient homomorphic integer computer from CKKS. *IACR TCHES*, 2025(4):873–898, 2025.
- [Kim25c] Jaehyung Kim. Faster homomorphic integer computer. *Cryptology ePrint Archive*, Report 2025/1440, 2025.
- [KN25] Jaehyung Kim and Taeyeong Noh. Modular reduction in CKKS. *CiC*, 2(2):17, 2025.

- [KSS24] Jaehyung Kim, Jinyeong Seo, and Yongsoo Song. Simpler and faster BFV bootstrapping for arbitrary plaintext modulus from CKKS. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2535–2546. ACM Press, October 2024.
- [LLNK22] Eunsang Lee, Joon-Woo Lee, Jong-Seon No, and Young-Sik Kim. Minimax approximation of sign function by composite polynomial for homomorphic comparison. *IEEE Transactions on Dependable and Secure Computing*, 19(6):3711–3727, 2022.
- [LM21] Baiyu Li and Daniele Micciancio. On the security of homomorphic encryption on approximate numbers. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 648–677. Springer, Cham, October 2021.
- [LMS24] Dongwon Lee, Seonhong Min, and Yongsoo Song. Functional bootstrapping for packed ciphertexts via homomorphic LUT evaluation. Cryptology ePrint Archive, Report 2024/181, 2024.
- [MCR21] Muhammad Haris Mughees, Hao Chen, and Ling Ren. OnionPIR: Response efficient single-server PIR. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2292–2306. ACM Press, November 2021.
- [PPS26] Jai Hyun Park, Alain Passelègue, and Damien Stehlé. THED: Threshold dilithium from FHE. Cryptology ePrint Archive, Paper 2026/638, 2026.
- [TLW⁺21] Benjamin Hong Meng Tan, Hyung Tae Lee, Huaxiong Wang, Shuqin Ren, and Khin Mi Mi Aung. Efficient private comparison queries over encrypted databases using fully homomorphic encryption with finite fields. *IEEE Transactions on Dependable and Secure Computing*, 18(6):2861–2874, 2021.
- [ZCL24] Yancheng Zhang, Xun Chen, and Qian Lou. Hebridge: Connecting arithmetic and logic operations in fv-style he schemes. In *Proceedings of the 12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC ’24, page 23–35, New York, NY, USA, 2024. Association for Computing Machinery.
- [ZWL⁺25] Xinxuan Zhang, Ruida Wang, Zeyu Liu, Binwu Xiang, Yi Deng, Ben Fisch, and Xianhui Lu. Phalanx: An fhe-friendly snark for verifiable computation on encrypted data. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’25, page 4035–4048, New York, NY, USA, 2025. Association for Computing Machinery.